

EVALUATING THE ROBUSTNESS OF GEMMA MODELS ON PERTURBED REASONING QUESTIONS: A COMPARATIVE STUDY

Ezgi BAKIR, Graduate School of Informatics, METU, Ankara, Turkey, ezgi.bakir@metu.edu.tr

Abstract- Large language models score highly on math reasoning benchmarks, but high scores do not always mean true understanding. This study tests how robust three open models from the Gemma family (Gemma 2 9B, Gemma 4 E4B, and Gemma 4 31B) are when reasoning questions are changed in small ways. A dataset of 100 questions was built by hand from Turkish central-exam questions. It is made of 25 base problems, each written in four forms: a standard version, an abstract version, a version with a distracting sentence, and an ill-posed version that has no valid answer. All questions were given in English with the same 8-shot prompt, and every solution was read and graded by hand. The results show a clear gap between model generations: total accuracy was 38% for Gemma 2 9B, 69% for Gemma 4 E4B and 87% for Gemma 4 31B. Adding a distracting sentence did little harm, while abstraction and impossibility were the hardest changes. Reading the full solutions also showed that accuracy alone hides weak reasoning, such as reaching a correct label through wrong steps or inventing rules to avoid declaring a problem is unsolvable. Hallucination was highest in the weakest model and almost absent in the strongest. The manual review also revealed some less expected behaviours: a model could read the same wording differently across versions of one problem, and in one case the same question produced different solutions on repeated runs even under greedy decoding.

Keywords—*large language models, mathematical reasoning, robustness, perturbation, hallucination, distractor robustness, ill-posed problems, Gemma models.*

I. INTRODUCTION

Large language models now score highly on many math and reasoning benchmarks. A high score, however, does not always mean that a model truly understands a problem. The model may have seen similar problems during training and may be matching familiar patterns instead of reasoning. Telling these two cases apart is difficult when a model is only tested on a fixed set of questions.

One way to test this is to change a question in small ways and check whether the model can still solve it. If a small change in the wording or the numbers causes the model to fail, this suggests that the model was relying on the surface form of the problem rather than on real understanding. Changes of this kind are called perturbations. In a perturbed question, the core problem or the mathematical idea stay the same, while the change is done on the surface for example by swapping the numbers, rewriting the wording, or adding an extra sentence. The point of a perturbation is to test whether a model really understands a problem or only recognises a familiar pattern from the training. If a model solves a question but then fails a small variant of it, then that is a sign of weak reasoning rather than true understanding.

This project uses that idea to compare three models from the Gemma family: Gemma 2 9B, Gemma 4 E4B, and Gemma 4 31B. For the study, a set of 100 reasoning questions was built by hand. Each question appears in four forms: a standard version and three perturbed versions. The perturbations make the question abstract, add a distracting sentence, or turn the problem into one that cannot be solved at all. This last type, the ill-posed question, is used to test something that standard benchmarks rarely check: whether a model can notice that a problem has no valid answer instead of forcing one.

The aim is not only to measure accuracy, but also to look at how each model reasons and where it fails. Because every solution was read by hand, the study can describe the kinds of mistakes each model makes, not just how many it makes. This closer reading makes it possible to compare the models across generations and sizes, to see which kinds of perturbation cause the most trouble, and to tell apart errors that look like ordinary slips from those that point to deeper weaknesses, such as inventing rules or changing the reading of a problem when it becomes hard. Taken together, these observations are used to ask a broader question: whether the models are truly reasoning about each problem or mainly responding to its surface form.

II. RELATED WORKS

Evaluating the robustness of large language models (LLMs) in mathematical reasoning typically relies on standard evaluation benchmarks. Most existing work in this domain builds on GSM8K, a popular dataset of grade school math word problems [1]. Many models score highly on it, often using chain-of-thought (CoT) prompting, where the model is shown worked examples and writes out its steps before the final answer [2]. However high scores on a fixed test set alone do not prove that a model reasons well, because the model might simply match familiar patterns seen during training.

To investigate this limitation, GSM-Symbolic was introduced [3]. The authors build symbolic templates from GSM8K so they can generate many versions of the same problem with new names and numbers. They find that accuracy changes significantly from one version to another, that performance drops as more clauses are added, and that adding a single irrelevant sentence causes the largest drop of all up to 65%. They read this as a sign that models rely on pattern matching rather than real reasoning. This paper was the main inspiration for the present study.

A related benchmark is GSM-Plus [4]. It extends GSM8K with eight types of perturbations, including changing numerical values, adding irrelevant information, and

altering the target of the question. Evaluating a wide range of language models, the authors found that mathematical reasoning is often not robust to such variations: models frequently fail modified versions of problems they can solve in their original form. The largest performance drops are observed on perturbations that require models to reconsider their assumptions rather than follow familiar solution patterns. More recent work has revisited these findings made some reinterpretations. Sturb [5] replicated the perturbation experiments on GPT-4o and Claude 4.5/4.6. Findings suggest that the reported performance drop largely disappears when ambiguous samples are filtered out. This suggests that models are not necessarily failing to reason. Instead, they may be making a reasonable judgment when a problem is genuinely underspecified.

Beyond changing the surface of solvable problems, some studies look at how models handle problems that cannot be solved at all. Sun et al. built a dataset of unanswerable math word problems and found that many models fail to notice that a question has no answer and instead hallucinate a solution [6]. Ma et al. reported a similar result: models can sometimes detect that a problem is unreasonable yet still produce hallucinated steps instead of stopping [7]. The tendency of models to be misled by extra, irrelevant information has also been studied directly, with Shi et al. showing that language models can be easily distracted by irrelevant context added to a problem [8]. These lines of work relate closely to the distraction and ill-posed question types used in the present study.

This study follows the same line but differs in a few ways. The questions are not taken from GSM8K. They are written from Turkish central-exam problems, with the scenarios and numbers changed. The perturbations are organised into four clear types, and one of them, the ill-posed type, goes beyond the usual benchmarks by asking whether a model can notice that a problem has no valid answer at all. Finally, every solution is read and graded by hand, which allows a closer look at how the models reason and fail, not only at their accuracy.

III. METHODOLOGY

A. Overview

For the goal of this project, firstly I built a set of 100 questions that each question belongs to one of four types. The first type is a normal question, and the other three types are perturbed versions that keep the same idea but change the surface form. I sent all 100 questions to three models and looked at both the final answers and the full solutions. As explained, this design follows the approach of Mirzadeh et al. (2024); only here it is applied to the Gemma models using a new, self-built dataset.

B. Dataset

A custom dataset that includes 100 questions was constructed for the project. The questions are based on word problems from Turkish central exams (the university

entrance exam and the military academy exam, MSÜ) from various years. These problems aim to test general problem solving and reasoning, not advanced math.

To avoid data leakage and copyright issues, the original exam problems were adapted by changing their scenarios and numerical values while preserving the underlying mathematical idea. For each question an English version and a Turkish version is written alongside with a full step-by-step solution in both languages. The experiments in this study use the English versions only. The models received English prompts and produced English answers. The full dataset, including the Turkish and English questions, the solutions, the final answers, and additional notes, can be accessed through the files in GitHub (link is at the end of the paper).

C. Question Types

For each problem, a standard version was created first and then three perturbed variants were generated:

- Type 1 — Standard: The normal version of the problem with concrete numbers. This is the baseline.
- Type 2 — Abstract: The numbers are replaced with symbols or abstract terms. The structure stays the same, but the model has to reason without concrete values.
- Type 3 — Distraction: A sentence with extra, unnecessary information is added. This sentence contains one or two numbers and is meant to confuse the model. A correct answer should ignore this sentence and the numbers in it.
- Type 4 — Ill-posed: These are the question that cannot be solved. For example, the problem asks for a positive and whole number of people or objects etc. but the math gives a decimal or a negative number, and the question also says the answer must be a positive integer which is impossible. The point is to see if the model notices that the question has no valid answer.

To make the four types clear, Table 1 has an example of a question family.

Type	Example
Type 1	There are three different departments in a company, and the total number of managed projects is 80. Either two or all three of these departments participated in each project. The number of projects in which each department took part is 64. Based on this, in how many projects did all three departments participate together?
Type 2	There are three different departments in a company, and the total number of managed projects is x . Either two or all three of these departments participated in each project. The number of projects in which each department took part is y . Based on this, what is the number of projects in which all three departments participated together, in terms of x and y ?
Type 3	There are three different departments in a company, and the total number of managed projects is 80. Half of the projects are internationally linked, and the average age of their employees is 32. Either two or all three of these departments participated in each project. The number of projects in which each department took part is 64. Based on this, in how many projects did all three departments participate together?

Type 4	There are three different departments in a company, and the total number of managed projects is 80. Either two or all three of these departments participated in each project. The number of projects in which each department took part is 40. Based on this, find the number of projects in which all three departments participated together as a positive integer.
--------	--

Table 1: Example for each question type from dataset.

In this example, Types 1 and 3 have the same numerical answer (32), Type 2 requires a symbolic expression ($3y-2x$), while Type 4 has no valid solution because the number of projects yields to a negative number which is restricted by the question text.

D. Models

The dataset was evaluated across three distinct models from the Gemma family that differ in size and generation. Gemma 2 9B and Gemma 4 E4B were run locally through Ollama, while Gemma 4 31B was accessed through the Google API in Google Colab due to hardware limitations. For consistency, all models were selected from the Gemma family. This allows the comparison to focus on differences in model size and generation while reducing the influence of differences in architecture and training approaches between model families. However, one important difference is that the 31B model was evaluated with reasoning mode enabled, whereas the local models were not. This limitation is discussed later in the limitations section.

E. Prompting

All three models used the same prompt. I developed more questions to use in 8-shot prompt. This means the prompt starts with 8 worked examples and then gives the actual questions. The 8 examples are two full sets of the four types, so the model sees a solved example of each type before it answers. The examples are fixed (hardcoded), not taken from the test set, so all 100 test questions could be used for evaluation. An additional zero-shot setting was not set; all results use the same 8-shot prompt. The full prompt can be accessed through the files in the same Github link in the end.

F. Experimental Setup

A Python pipeline was developed to automatically send prompts, collect model responses, and store outputs for later analysis. Questions and reference solutions were stored in a spreadsheet and processed sequentially. Responses were saved together with the original question metadata for evaluation.

The main generation settings were the same for all models:

- Greedy decoding (temperature = 0), so the output is deterministic.
- A large enough output limit (about 2048 to 8192 tokens) so long answers are not cut off.
- For the local models, the context window was set to 8192 tokens, because the long 8-shot prompt does

not fit in Ollama's default size and was being cut off silently.

- Basic retry and error handling, so failed or empty answers were retried and never left blank in the output file.

Each question was asked once. Because decoding was greedy, asking the same question again normally gives the same answer. In a few cases it was noticed that a small mistake in the question or its translation. I fixed those questions and ran them again. For some of them the answer changed and for some it did not. These examples and regarding observations will be examined in the discussion part.

G. Evaluation

The pipeline did compute an automatic exact match accuracy from the extracted final answers. However, it is not used for the results in this report because it focuses only on the final answer and does not show whether the reasoning process was correct or not. Instead, I read every full solution and evaluated the responses just like evaluating an exam paper of a student. During this review, common response patterns were noted and responses grouped into categories such as correct answer with correct solution, wrong answer with correct solution, wrong answer with wrong solution, incomplete solution with correct/wrong answer, distraction is noticed and addressed/not noticed, ill posed detected/not detected, etc. These categories were not defined in advance. Instead, the responses were first reviewed to identify recurring and unexpected patterns, and the categories were then developed from these observations. This is an inductive coding approach which means the categories come from the data. The categories are not exactly same for each type and will be examined in the results section in details.

IV. RESULTS

The overall accuracy of the three models for the four question types is shown in Table 2. Each cell gives the accuracy for one model to one question type, measured from the manual review. Correct solution with a correct final answer or, for the ill-posed type, the "impossible" statement. Also note that the extracted answers are excluded from the analysis because in many cases, the extracted answer did not match the answer that appeared at the end of the model's solution.

Model	Type 1 (Standard)	Type 2 (Abstract)	Type 3 (Distr.)	Type 4 (Ill- posed)	Overall
Gemma 2 9B	44%	32%	40%	36%	38%
Gemma 4 E4B	72%	64%	72%	68%	69%

Gemma 4 31B	96%	88%	92%	72%	87%
-------------	-----	-----	-----	-----	-----

Table 2: Overall accuracy percentages of each model on each question type.

These accuracy values give a summary of how the models performed, but accuracy on its own is not the focus of this study. For each model, attention was also how the solution was reasoned, where the mistakes appeared, and how the model tend to respond to the change introduced in that question type. The study also aimed to examine whether the models repeat certain behaviours in specific question types and whether there are any common patterns in these behaviours. In the sections below, the results for each question type are reported separately, and within each type the three models are discussed in the order starting with Gemma 2 9B, followed by Gemma 4 E4B, and ending with Gemma 4 31B.

As I expected, the lowest performance was observed on Type 2 (abstract) questions for all models, except Gemma 4 31B, in which the lowest performance occurred on Type 4 questions. One possible reason might be that abstract questions remove the numerical clues that models often rely on and force them to reason about relationships and structures alone.

A. Results on Type 1 (Standard Questions)

The standard questions are the baseline. The results here are mainly used as a reference point for the perturbed types in the following sections.

Gemma 2 9B: On the standard questions an accuracy of 44% was reached. A correct solution with a correct final answer was produced in 40% of the cases, and in one further case (4%) the final answer happened to be correct even though the solution behind it was a bit flawed. A wrong solution leading to a wrong answer was seen in 36% of the questions, and among these wrong answers 2 false negatives were observed: For one of these questions, model said the question was impossible to solve and for the other one it stated that there was insufficient information which was not the case. 20% of the answer was left incomplete or no final answer was given. And finally hallucinated reasoning was found in 8% of the answers.

Gemma 4 E4B: Accuracy on the standard questions observed as 72%, with a correct solution and a correct answer. A wrong solution with a wrong answer occurred in 12% of the questions, while an incomplete answer caused by the token limit was seen in 8% and a failed (empty) generation in 4%. Also it should be noted that incomplete and generation failed responses were more common in the first run with Gemma 4 E4B. For these particular cases, I sent the same questions to the model again and used the new responses. As a result, the counts reported here are improved than what I observed in the initial outputs. In one case (4%) the solution was correct, but the final answer was wrong. The hallucination rate was 8%. In these cases,

model was observed to make intermediate assumptions that were not supported in the question.

Gemma 4 31B: This model reached 96% accuracy, with 24 of the 25 standard questions answered correctly, and no hallucinated reasoning was observed. The error in a single question was caused by the model interpreting the question statement differently than intended. The model read the phrase "current number of female students" as the number before new students join, while what I intended was the number after they join. The calculations were correct at every step, and the model computed the correct value (new value) in its internal check, but still reported the previous number of students. It also read the same phrase the same way in the other versions of this family, so this was a consistent reading of an ambiguous wording rather than a mistake. If this answer is accepted, the model's accuracy on the standard questions is 100%.

B. Results on Type 2 (Abstract Questions)

Gemma 2 9B: The abstract questions were the hardest type for this model, with an accuracy of only 32%. A fully correct solution and answer were given in 28% of the cases, with one more correct answer (4%) reached from a wrong solution. A wrong solution with a wrong answer was the most common outcome at 48%, and in another 20% the answer was incomplete or still contained an unknown that the model invented by its own. The hallucination rate climbed to 20% in which model came up with some formula or invented logic to fill the reasoning gaps in solutions. Two failure patterns were specific to this type. In several answers a helper variable or unknown was produced but in the final answer model fail to simplify it and left inside the final answer which caused the solution to be classified as wrong. In about 20% of the cases, the model changed the meaning of a variable during the solution. This behaviour was observed almost exclusively in the abstract questions.

Gemma 4 E4B: Here an accuracy of 64% was found on the abstract questions. A correct solution with a correct answer was produced in 28% of the cases, and in the remaining 36% the solution and the answer were both correct even though the automatic extractor pulled the wrong value out of the text. This type shows clearly why the manual review was needed: the automatic score would have reported only 28% for this model, which would have badly understated its real performance on detailed answers. Also, incomplete answer caused by the token limit was seen in 20% of the questions, a wrong solution with a wrong answer in 12%, and one answer (4%) was wrong because of a misreading in the final step. The hallucination rate was 8%. A small number of these questions were sent to the model more than once after translation fixes, and the repeated runs were not stable: in one case the same question moved from an incomplete answer to a correct answer with no working shown, and finally to a fully correct solution. The implications of these different takes will be further investigated in the discussion section.

Gemma 4 31B: Accuracy on the abstract questions was 88%. In most cases, the model's answer matched the

reference solution exactly. One additional answer was also counted as correct because the model solved the problem in a more general form and when simplified the result becomes mathematically valid. No hallucinated steps were found at all, which stands in sharp contrast to Gemma 2 9B, whose hallucination rate on this same type was 20%; here the algebra stayed honest from start to finish. Only three questions were answered incorrectly. One of them was a generation failed case. The other two errors came from the model misunderstanding a word in the question and a translation issue. These mistakes were not related to reasoning ability, but rather to difficulties in understanding the language.

C. Results on Type 3 (Distraction Questions)

For this type, two things were recorded: whether the model was fooled by the distracting sentence, and whether the model noticed the distracting information at all.

Gemma 2 9B: Accuracy on the distraction questions was 40%. A correct solution with a correct answer was given in 40% of the cases, with one near-correct answer (4%) spoiled by a single slip, while a wrong solution with a wrong answer occurred in 44% and an incomplete answer in 12%. The model was actually misled by the distracting sentence in only 16% of the questions; for the rest the extra information was ignored. However, in none of the cases the model explicitly say that the extra information was irrelevant. The distractor was dropped silently rather than identified. The hallucination rate was 12%, and on a few occasions the distracting sentence pushed the model into a false "impossible" label or into giving no answer at all.

Gemma 4 E4B: Accuracy on the distraction questions was 72%. In 40% of the cases, the model gave the correct answer and clearly pointed out the distracting information. In another 32%, it again gave the correct answer but did not mention the distractor, while still not being affected by it. The model was misled into a wrong answer in 8% of the questions, and an incomplete answer was given in another 8%. Of the remaining three responses, two recognized the distraction but still arrived at an incorrect solution. In the third case, the model was again confused by an ambiguity introduced during translation, leading to a different interpretation of the problem and an incorrect answer. The hallucination rate was 4%.

Gemma 4 31B: Accuracy on the distraction questions was 92%. The model was never once misled by the distractor (0%), and in 76% of the cases the extra information was explicitly named as a distractor that did not affect the calculation. In the remaining correct cases the distractor was simply set aside without comment. No hallucination was found. The two wrong answers were a question cut off by the token limit, where the setup was already correct, and one case affected by an ambiguous wording in the dataset. It is worth adding that every distraction question that had fooled Gemma 2 9B was answered correctly by this model.

D. Results on Type 4 (Ill-Posed Questions)

For this type, an answer was counted as correct only when the question was identified as impossible or unsolvable.

Gemma 2 9B: Accuracy on the ill-posed questions, measured by the correct "impossible" label, was 36%. Very importantly only 12% of the answers reached this label through correct reasoning. Another 24% were "lucky impossible" cases, where the model gave the correct label but for the wrong reasons, often based on incorrect equations. This means that most of the correct labels were not properly justified. In one case (4%) the impossibility was detected and then overridden to produce an answer anyway. A wrong solution that missed the impossibility and gave a number was the most common outcome with 48%. This type had the highest hallucination rate, at 36%. In 16% of the answers, the model rounded the result in a way that was not allowed. The rounding was inconsistent and was always used to force a whole number answer.

Gemma 4 E4B: The impossibility was detected correctly in 68% of the ill-posed questions. The model failed to notice it in 12% of the cases, gave an incomplete answer in 8%, and returned an empty generation in another 8%. No lucky impossible cases were observed in the Type 4 responses here. The model did not classify any problem as impossible through incorrect reasoning. Also, the hallucination rate was 8%. A few of the incomplete cases were sent again, and on the second run the question was correctly identified as impossible.

Gemma 4 31B: The impossibility was detected correctly in 72% of the ill-posed questions, and in every one of these correct cases the real contradiction in the problem was found rather than guessed. The "lucky impossible" pattern seen in Gemma 2 9B, where a correct label was reached from wrong reasoning, also did not occur here at all. The unauthorised rounding that was typical of Gemma 2 9B was also absent: when a count came out as a fraction, the model stopped at the fraction and declared the question impossible. No hallucination was found. Of the remaining questions, three were missed because of issues in the dataset and four returned a generation error.

E. Results on Hallucination

Additionally, one more pattern was tracked across all types and is reported here as a separate measure: the hallucination rate. These rates are shown in Table 3 below.

Model	Type 1 (Standard)	Type 2 (Abstract)	Type 3 (Distr.)	Type 4 (Ill-posed)	Overall
Gemma 2 9B	8%	20%	12%	36%	19%
Gemma 4 E4B	8%	8%	4%	8%	7%
Gemma 4 31B	0%	0%	0%	0%	0%

Table 3: Hallucination rates in solutions

Because hallucination is reported here as a separate measure, it needs a clear definition. In this study a hallucination was counted whenever the model introduced a rule, a formula, a premise, or a numerical value that the question did not contain and that could not be derived from it. This is different from an ordinary mistake. A wrong setup or an arithmetic slip is still an attempt to work with the information that was given, while a hallucination adds information that was never there. The distinction was needed because plain accuracy treats both kinds of error in the same way, even though they point to very different things: a slip is a failure of execution, but a fabricated rule is a failure to respect what the problem allows. Several types were observed. Most of them were invented logic rather than invented facts, such as made up rounding rules used to turn a fractional or impossible result into a whole number. Also fabricated equations or formulas built to fill a gap in the reasoning, and unsupported assumptions inserted in the middle of a solution and considered as if they were standard for such problems.

As seen in Table 3 above, hallucinations became less frequent as the models improved. Another clear pattern was that hallucinations occurred most often in the Type 4 (ill-posed) questions. One possible reason is that hallucination often acted as a way for the model to avoid admitting that a problem could not be solved cleanly. This happened most often in the ill-posed questions, where a correct solution would have required stating that there was not enough information or that no unique answer existed. It also appeared, though less frequently, in the abstract questions, where there were no concrete numbers to guide the reasoning. Instead of stopping, weaker models often made up a rule, formula, or assumption that allowed them to produce a neat final answer. Some examples are shown here from Gemma 2 9B below illustrate these behaviours:

Example 1: Normalizing an impossible condition

Question: “Two cars set off at the same time to travel from city A to city B. The first car travels at a speed of 80 km/h, and the second car travels at a speed of 120 km/h. Given that the first car arrives at city B 3 hours earlier, what is the distance in km between cities A and B?”

Normally, it is impossible for slower car to reach earlier than the faster car, which makes the problem ill-posed. A correct equation would make the first car's time 3 hours less than the second's, $d/80 = d/120 - 3$, which gives a negative distance and confirms the contradiction. The model instead wrote: “We know the second car arrives 3 hours earlier, so: $d/80 = d/120 + 3$ ” and solved it cleanly to $d = 720$. The model used a premise the question never stated and quietly reversed the stated condition to turn an impossible problem into a solvable one. The arithmetic was perfect; the failure was in accepting a contradictory premise by silently rewriting it.

The same impossible question was given to Gemma 4 E4B, and its answer is interesting for a different reason. This model did notice the contradiction. It stated that the slower

car cannot arrive earlier than the faster one and then wrote: “Assuming this is a typo and the intended meaning is that the difference in arrival time is 3 hours, we set up the equation: $T_1 - T_2 = 3$ ”. It then solved this cleanly and reached 720 km, the same wrong answer as Gemma 2 9B. This shows that the two models failed in different ways. Gemma 2 9B silently reversed the condition without seeming to notice anything was wrong, while E4B saw the contradiction, named it, and then decided on its own that the question must contain a typo and repaired it. The final result was the same, but only the second model showed any awareness that the problem was broken.

Example 2: Rounding instead of stating “impossible”

Question: “An intelligent irrigation system in an olive grove runs for a fixed duration each day, allowing the entire grove to be watered in 40 days using the water in the reservoir. If the daily irrigation duration is shortened by $1/7$, for how many days can the grove be watered with the same amount of water? Find the answer as a positive integer.”

The correct solution yields to an answer 46.7 which makes the question ill-posed because in the question it is explicitly stated that the answer should be a positive integer. The model reached the right value and then wrote: “Since we need a positive integer answer, round up to the nearest whole number: 47”. The model invented a rounding rule to turn an impossible result into a valid answer and failed to check whether the condition was actually satisfied.

This invented rounding issue was observed in 4 of the 25 ill-posed questions (16%) of Gemma 2 9B solutions. Each time the model reached to a value, a fractional count of people or objects, or a decimal where the question asked for a whole number. Then it did not stop and call the problem impossible but instead it rounded the value on its own and reported the rounded number as the answer. The direction of the rounding was not consistent either; the model rounded down in one case and up in others. This is counted as invented logic, because the rounding rule was never given in the question and was added only to force the result. It is one of the main reasons the hallucination rate was highest on the ill-posed type for two models.

Example 3: A hallucination from Gemma 4 E4B to a type 2 (abstract) question:

Question: “An intelligent irrigation system in an olive grove runs for a fixed duration each day, allowing the entire grove to be watered in “A” number of days using the water in the reservoir. If the daily irrigation duration is shortened by $1/x$, for how many days can the grove be watered with the same amount of water?”

The question is the abstract version, with A and x instead of numbers. The model started well. It set the total water as $W = (A \cdot R_initial)$ and said the new number of days is the total water divided by the new daily rate. Then two problems appeared. First, when the duration was shortened by $1/x$, the model said this as a plain subtraction, $R_new = (R_initial) - (1/x)$, instead of scaling the rate to $(1 - 1/x)$ of the original. Second, and more importantly, it got stuck

because its answer still contained R_{initial} , which the question never gave. Then it could not finish in terms of A and x alone. At this point the model wrote that *"in problems of this nature it is common to assume that $R_{\text{initial}} = 1/A$ "*, and used this to keep continue. No such rule exists, and the question never stated it. The model invented an outside assumption, and even called it common practice, only to escape the point where it was stuck and reach an answer. The output then ran into the token limit before finishing, that's why this case was labelled both as a hallucination and an incomplete answer.

V. DISCUSSION

A. The Overall Picture

The three models did not only differ in accuracies. They also differed in how they handled each kind of perturbation. Across all four types the ordering was the same: Gemma 4 31B was clearly the strongest, Gemma 4 E4B was in the middle, and Gemma 2 9B was the weakest. The more interesting question is which perturbation hurt which model more.

Among the three perturbations, the distraction questions caused the least damage, while the abstract questions and the ill-posed questions caused the most. It helps to keep these two kinds of change apart as results suggested. The distraction questions add extra words but leave the underlying problem untouched, so can be considered as a surface change among all. The abstract questions change how the problem must be represented, because the model can no longer lean on concrete numbers and have to work with symbols instead. The ill-posed questions change whether the problem can be solved at all. The results show that the surface change was the easy one to survive, while the change of representation and the change of solvability were not.

As shown on Table 1, Gemma 2 9B dropped only from 44% on the standard questions to 40% on the distraction questions, but it fell to 32% on the abstract questions. When it comes to the ill-posed questions the table shows a 36% accuracy but in fact only 12% was scored by valid reasoning rather than by the final label alone (in other words when we exclude the "lucky impossible" cases). Almost a similar situation appears for Gemma 4 E4B too, whose lowest score among the four types was on the abstract questions. For the 31B model the picture was different: it stayed at or above 88% on three of the four types, and only the ill-posed type pulled it down to 72%.

B. Gemma 2 9B: How It Fails

A close reading of Gemma 2 9B's answers from Table 1 and 2, the difficulty ranking by type was Standard (44%) > Distraction (40%) > Ill-posed (36% by label) > Abstract (32%), so abstraction was the hardest of the solvable types and the ill-posed questions were where the model both failed hardest and fabricated most.

The most consistent pattern I observed across the results is that hallucination rates rose as model performance declined. The rate climbed from 8% on the plain questions to 12% on the distraction questions, 20% on the abstract questions, and 36% on the ill-posed questions. Most of these were invented rules or invented logic, not invented facts. When a contradiction blocked the normal path to a solution, the model usually did not stop but instead it found a way around the problem. Sometimes it quietly rewrote the contradictory part into something solvable, or made up a rounding rule, or sometimes it built a different equation that hid the conflict. In other words, the hallucination were an escape behaviour. It showed up most often in the exact cases where a correct answer would have meant admitting that no answer exists.

This is also why the model's "impossible" label cannot be trusted on its own. When we look at the final label, detection looked like 36%, but scored by valid reasoning it was only 12%, because most of the correct labels came from wrong equations (the rounding behaviour behind some of these is shown in Section 5.E). The opposite of this also happened: on about 9% of the solvable problems the model built a wrong equation, reached a dead end, and then blamed the question by calling it impossible or lacking information. So the label fails in both directions. When the model says "impossible," the problem is often solvable, and when it solves a problem, the result is often reached for the wrong reasons. The label tells us little about whether the model actually understood the problem.

Another pattern is worth noting. The model was surprisingly resistant to distraction (it ignored the irrelevant sentence in about 84% of the cases independent from producing a correct or wrong solution), but the resistance was silent: it almost never stated that the extra information was irrelevant, it simply dropped it. Also the model applied almost no sanity checking to its own output. Answers such as a negative distance, a grandchild older than the grandparent, occupied seats exceeding the capacity, or an unknown left sitting inside the "final answer" were handed in without any sign that the model noticed they could not be real.

Another observation about Gemma 2 9B's performance was that most of the unfinished answers were not cut off by the token limit. In 9 of the unfinished solutions, the model stopped on purpose. In six cases it ended with the line *"let me know if you'd like me to work through the solution."* In three more it did the same in other words, telling the reader to *"solve for x "* or saying the answer *"can be expressed in terms of A , B , and x "* without finishing. The sentence was complete, but the model just chose to stop. What these cases share is the kind of work that was left, not the topic of the question. The model set up the equations correctly, but then it stopped right at the point where it needed to solve them. If only one calculation remained, it finished the problem. But if the next step was a system of two equations or a long substitution, it stopped there and left that part to the reader. Writing the plan seemed to be enough for the model. This is not a real ending for a math problem, and it again

suggests that the model treated the task like a conversation, not a problem with one definite answer.

C. Why the Models Differ: Architecture and Training

The accuracy gap between Gemma 2 9B and the two Gemma 4 models was expected, and it is best explained by the distance between the two generations. Gemma 2 was released in 2024 [9], while Gemma 4 was released in 2026 [10]. Between these two generations, Gemma models gained several important improvements, such as better multimodal capabilities, longer context windows, and support for more languages. Because of these changes, it is not surprising that the newer Gemma 4 E4B model was able to match or outperform the older Gemma 2 9B model even though it has smaller effective size. One possible explanation is improvements in training and distillation. Both model families were trained using knowledge distillation, where a larger model helps train a smaller one [9]. However, the Gemma 4 family was developed from a more advanced research base and benefited from newer training methods and data [10]. These improvements may have contributed to the higher accuracy and lower hallucination rates observed in the Gemma 4 models, although this study cannot determine the exact contribution of each factor.

Another important factor is reasoning ("thinking") mode. This may be one of the most important differences for this task. Gemma 4 models include a configurable thinking mode that allows them to reason step by step before producing an answer [10], [11] while Gemma 2 does not. In this study, the 31B model was tested with thinking mode enabled, whereas the two local models without it. This may help explain some of the behaviours that distinguished the 31B model. For example, when it correctly identified an ill-posed question, it usually explained the actual contradiction rather than simply giving the correct label. It also identified distractors more often and was less likely to produce invented values or unsupported calculations. However, this also makes the comparison less fair, because the 31B model was the only one tested with the thinking mode on. The two smaller models were run without it. So we cannot fully separate how much of the 31B's advantage comes from the model itself and how much comes from this extra reasoning step. This limitation is discussed further in the Limitations section.

Another factor worth examining is the design of the smaller models. The E2B and E4B models do not use a Mixture-of-Experts architecture. Instead, they use a design called Per-Layer Embeddings (PLE), which increases the model's representational capacity without a large increase in computation [11]. The 31B model, by contrast, is a dense model that uses all of its parameters for every token.

These differences help explain the results in two ways. Within the Gemma 4 family, the gap is mostly about size: the dense 31B is much larger than the 4B-effective E4B, so it is expected to be stronger. However size alone does not explain everything. Gemma 2 9B has more parameters than

E4B but still performed worse, so its weakness comes from its generation rather than its size, maybe, from weaker training and the lack of the newer architectural features. The efficient PLE design may be one reason E4B performed above its small size and even outperformed the older, larger Gemma 2 9B.

D. Human vs. Machine Errors

Because this project sits within a course on reasoning and cognition, the kind of mistakes the models made is also interesting. Almost all of Gemma 2 9B's errors were typical machine mistakes: it presented circular equations as answers, it overwrote its own correct intermediate results a few lines later, it normalised a contradictory problem into a familiar template, and it treated "produce a number" as something it had to do even when no number existed. Several times it contradicted a statement it had made earlier in the same solution, which a human would rarely do. Out of the full set, only a single error looked genuinely human: one solution that was correct except for adding where it should have subtracted, the sort of slip a tired student makes. Out of the full set, only a few errors looked genuinely human. One was a solution that was correct except for adding where it should have subtracted, the sort of slip a student would make. In another solution, the model set up the logic and the equation correctly but then made a mistake in the algebra and reached the wrong result. In another one while finding a class average, the model used the old class size in the final step and forgot that the number of students had increased, so it divided by the wrong total. As far as I observed, only these solutions can be considered human or student-like mistakes.

Results might suggest that, at least for the weakest model, wrong answers do not come from human-like misreadings or miscalculations but from a lack of any check that the reasoning stays consistent and that the final answer could be true in the real world. The stronger Gemma 4 models show much less of this, which might prove the idea that better training and an explicit reasoning helps the model to the kind of self-consistent, sanity-checked reasoning that the weaker model lacks.

E. Outcomes of Different Takes

Because Gemma 4 31B made so few errors, its mistakes were easy to study one by one. This close reading showed me something the first two models had hidden: a few of my own questions had wording that could be read in more than one way. The interesting part is how the model handled them.

In one question, I wrote "A lira" meaning A as a variable, but the English version could also be read as "one lira." This family has four versions that all use the same phrase. The model 31B read it correctly in two of them and solved them right. But in the other two, the abstract and the ill-posed versions, it read the phrase as "one lira" and solved them wrong. If the wording were simply broken, all four should have failed. Instead, only the harder versions did.

There is one more example of an ambiguous wording is the problem. Here the phrase "60 cargo containers per truck driver and 15 per distribution staff" can be read in two ways: either the two groups share one common pool of containers, or each group handles its own separate amount. The intended reading is the shared pool, and this is the reading that makes the ill-posed version unsolvable. In the first three versions the model used exactly this intended reading, set up the shared pool equation, and solved them correctly. However, the ill-posed version has no whole number answer under that same reading. At this point the model did not report the problem as impossible. Instead, it switched to the other reading, set up the separate equation, and that version happened to give a clean integer. In other words, the model read the phrase one way in the three solvable questions, and then, only when the intended reading led to an impossible result, it changed to the reading that produced a tidy answer. The wording never changed; the model's reading changed once the problem became unsolvable.

Finally, there was another important observation. While working with Gemma 4 E4B, the same abstract question ended up being sent three times. The first time, it was sent as part of the normal run. The scores on the abstract questions were low, so I sent some type 2 questions once, so this question was sent a second time. Separately, I also re-sent the answers that had come back incomplete. This question's first answer was incomplete, so it fell into that group too and was sent a third time. I did not notice at first that the two retake had overlapped on the same question. The interesting thing is that, the question was solved differently each time. In the first run the solution was started correctly but left incomplete. In the second run there was no working at all, only a correct final answer. In the third run both the solution and the answer were correct.

This is a clearer example of a genuine "roll of the dice." Unlike the family cases above, here the input did not change at all; the very same question produced three different outputs. This is striking because all runs used greedy decoding (temperature 0), which should give the same answer every time, but the model's output was not fully stable even for an identical prompt. This connects directly to Mirzadeh et al. (2024), who reported that model performance varies across repeated trials of the same problem; our case shows the same kind of variation, here on a single question sent three times accidentally.

VI. LIMITATIONS

This study has several limitations that should be kept in mind when reading the results.

First, the dataset is small. It has 100 questions built from 25 base problems, so each question type has only 25 items. With numbers this small, a few questions can move a percentage by a lot, and the results should be read as trends rather than exact figures.

Second, each question was sent only once, and decoding was greedy. As the E4B example showed, the output was

not always stable even for the same prompt, so a single run does not fully capture a model's behaviour. Running each question, several times would give a more reliable picture.

Third, the three models were not run under identical conditions. The two smaller models ran locally through Ollama, while the 31B model ran through an API with its reasoning mode left on. So the strongest model also had an extra reasoning step that the others did not, which makes the comparison less than perfectly fair.

Fourth, the grading was done by hand. Reading every solution allowed a deeper analysis, but it also brings some subjectivity, especially when deciding whether reasoning was valid or whether a step counted as a hallucination. Contribution of other independent graders would make these judgments stronger.

Fifth, some questions had wording that could be read in more than one way, and a few translation choices affected how the models understood them. These cases were noticed only during the manual review, and they show that the questions themselves were not perfectly controlled. At the same time, these cases turned out to be useful in an unexpected way. Looking at how a model dealt with an ambiguous wording, especially when it kept one reading across easy questions but switched to another on the hard version, told us more about how the model works than a clean question would have. So while this lack of control is a limitation, it also opened up some of the more interesting observations in this study.

Finally, the experiments used only the English version of each question and also with the same 8-shot prompt. A Turkish version of every question was written and is available in the dataset, but it was not tested. Running the models on the Turkish wording, or with a different prompt or a zero-shot setting, could lead to different results, and comparing the two languages is left for future work.

VII. CONCLUSIONS AND FUTURE WORK

This study tested how three Gemma models handle reasoning questions when the questions are changed in small ways. Each of 25 base problems was written in four versions: a standard one, an abstract one, one with a distracting sentence, and an ill-posed one with no valid answer. The 100 questions were given to Gemma 2 9B, Gemma 4 E4B, and Gemma 4 31B, and every solution was read and classified by hand.

The results were clear. The newer Gemma 4 models were much stronger than the older Gemma 2 9B, and the 31B model was the strongest by a wide margin. Adding a distracting sentence did little harm, while abstraction and impossibility were the hardest changes for the weaker models. The manual reading also showed that accuracy alone hides some important information such as the weakest model often reached a correct label through wrong reasoning, invented rules to avoid saying a problem was impossible, and left answers unfinished by handing the work back to the reader. The strongest model showed

almost none of this, but even it could shift its reading of a question once the problem became unsolvable. Taken together, these patterns point to behaviour that depends on the surface form of a problem more than on a stable understanding of its meaning.

Future work could extend this in several directions. The dataset could be made larger to give more reliable numbers, and each question could be run several times to measure stability. The Turkish versions of the questions, which already exist, could be tested to compare performance across languages. The models could also be compared under matched settings, for example with the reasoning mode turned on or off for all of them, to make the comparison fairer. Beyond this, new kinds of perturbations could be designed to challenge stronger reasoning models, since the simple distractions used here barely affected them; harder or more deceptive changes might reveal weaknesses that these questions did not. Finally, my own experience as a teacher suggests that the mistakes students make are often different from the ones observed here, and this contrast is what gave me the idea for my future work: to compare model errors directly with the mistakes humans make on the same questions.

DATA AND CODE AVAILABILITY

The dataset, the 8-shot prompt examples, the raw model outputs for all three models, and the Python pipeline used to run the experiments are publicly available at:

<https://github.com/EzgiBakir/COGS-560-Project-LLMs-with-Perturbations/blob/main/README.md>

REFERENCES

- [1] K. Cobbe et al., "Training verifiers to solve math word problems," arXiv:2110.14168, 2021.
- [2] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 24824–24837.
- [3] I. Mirzadeh et al., "GSM-Symbolic: Understanding the limitations of mathematical reasoning in large language models," arXiv:2410.05229, 2024.
- [4] Q. Li, L. Cui, X. Zhao, L. Kong, and W. Bi, "GSM-Plus: A comprehensive benchmark for evaluating the robustness of LLMs as mathematical problem solvers," in *Proc. 62nd Annu. Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 2961–2984.
- [5] "Revisiting GSM-Symbolic: Do 2026 frontier models still fail at confounded grade-school math?," *LessWrong*, 2026. [Online]. Available: <https://www.lesswrong.com/posts/Zc4C99Dasj74YKCFh/>
- [6] Y. Sun, Z. Yin, Q. Guo, J. Wu, X. Qiu, and H. Zhao, "Benchmarking hallucination in large language models based on unanswerable math word problem," in *Proc. 2024 Joint Int. Conf. Computational Linguistics, Language Resources and Evaluation (LREC-COLING)*, 2024, pp. 2178–2188.
- [7] J. Ma, D. Dai, Z. Yuan, R. Li, W. Luo, B. Wang, Q. Liu, and L. Sha, "Large Language Models Struggle with Unreasonability in Math Problems," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2026, pp. 32428–32436. doi: 10.1609/aaai.v40i38.40518.
- [8] F. Shi et al., "Large language models can be easily distracted by irrelevant context," in *Proc. 40th Int. Conf. Machine Learning (ICML)*, 2023, pp. 31210–31227.
- [9] Gemma Team, "Gemma 2: Improving open language models at a practical size," arXiv:2408.00118, 2024.
- [10] Google, "Gemma 4 model card," Google AI for Developers, 2026. [Online]. Available: https://ai.google.dev/gemma/docs/core/model_card_4
- [11] Google, "Gemma 4 model overview," Google AI for Developers, 2026. [Online]. Available: <https://ai.google.dev/gemma/docs/core>